Report on working with project "House Prices — Advanced Regression Techniques" (Kaggle competition)

Author: Vsevolod Muronets

The chosen project is Kaggle's competition "House Prices — Advanced Regression Techniques". The goal of the project is to build a model that will most precisely predict the sale prices of houses in the test dataset using machine learning techniques. The suggested machine learning model to use is TensorFlow Decision Trees, but since it was not covered in our course, I use what we have already discussed (by the time of committing Intermediate Report) — various regression models (Linear, Lasso and Ridge ones) from Scikit-learn (Sklearn) library with "SalePrice" as the dependent variable and other variables as independent. It may provide less correct predictions than TensorFlow Decision Trees (although that is yet to be tested, however), but will constitute a better learning practice.

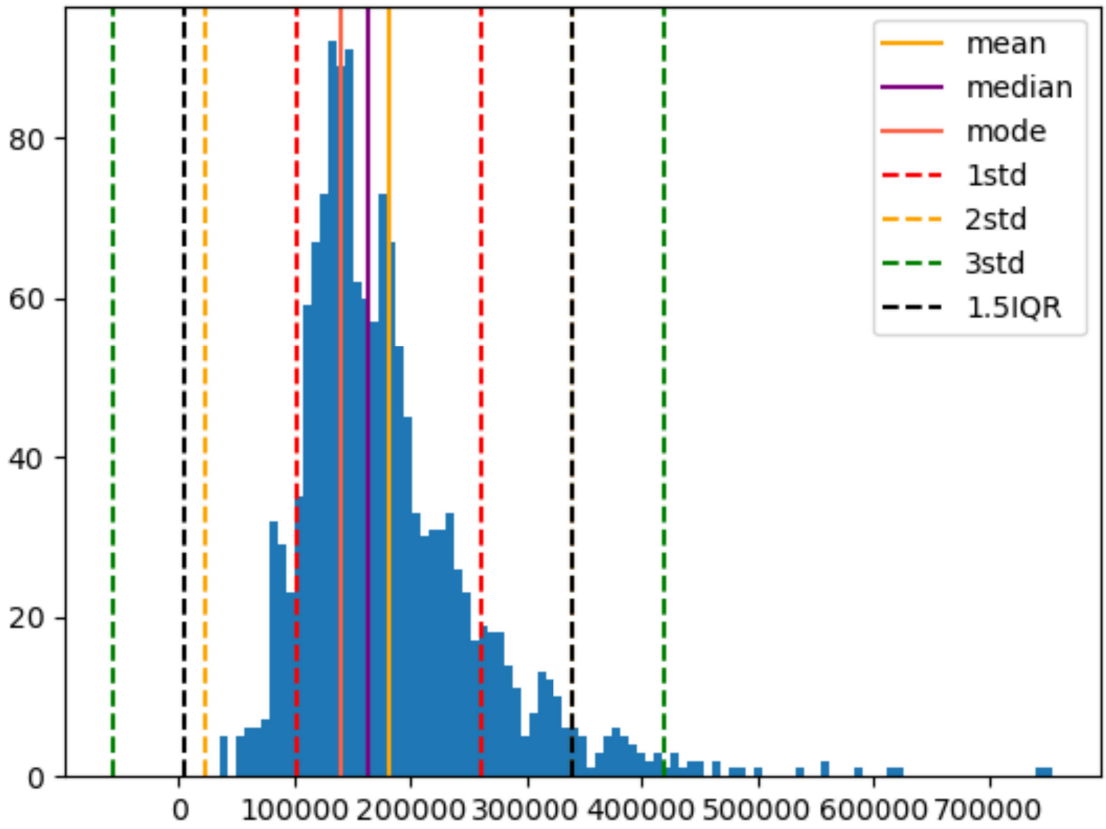
To begin with the project, I load the training dataset as a Pandas DataFrame and look at its features. The training data contains 1460 observations (rows) and 81 features. As can be seen from the table below (Table 1), 38 of them are (at least seemingly) numerical, while other 43 are categorical (that is, of "object" datatype). The column 'Id' contains ID's of observations and is of no use for regression modeling. Thus, this column should be excluded from further analysis, so I drop it from the training dataset that I will work with. 37 numerical variables remain.

Variable name	Value type	Variable name	Value type
Id	int64	**CentralAir**	object
MSSubClass	int64	**Electrical**	object
MSZoning	object	**1stFlrSF**	int64
LotFrontage	float64	**2ndFlrSF**	int64
LotArea	int64	**LowQualFinSF**	int64
Street	object	**GrLivArea**	int64
Alley	object	**BsmtFullBath**	int64
LotShape	object	**BsmtHalfBath**	int64

LandContour	object	**FullBath**	int64
Utilities	object	**HalfBath**	int64
LotConfig	object	**BedroomAbvGr**	int64
LandSlope	object	**KitchenAbvGr**	int64
Neighborhood	object	**KitchenQual**	object
Condition1	object	**TotRmsAbvGrd**	int64
Condition2	object	**Functional**	object
BldgType	object	**Fireplaces**	int64
HouseStyle	object	**FireplaceQu**	object
OverallQual	int64	**GarageType**	object
OverallCond	int64	**GarageYrBlt**	float64
YearBuilt	int64	**GarageFinish**	object
YearRemodAdd	int64	**GarageCars**	int64
RoofStyle	object	**GarageArea**	int64
RoofMatl	object	**GarageQual**	object
Exterior1st	object	**GarageCond**	object
Exterior2nd	object	**PavedDrive**	object
MasVnrType	object	**WoodDeckSF**	int64
MasVnrArea	float64	**OpenPorchSF**	int64
ExterQual	object	**EnclosedPorch**	int64
ExterCond	object	**3SsnPorch**	int64
Foundation	object	**ScreenPorch**	int64
BsmtQual	object	**PoolArea**	int64
BsmtCond	object	**PoolQC**	object
BsmtExposure	object	**Fence**	object
BsmtFinType1	object	**MiscFeature**	object
BsmtFinSF1	int64	**MiscVal**	int64
BsmtFinType2	object	**MoSold**	int64
BsmtFinSF2	int64	**YrSold**	int64
BsmtUnfSF	int64	**SaleType**	object
TotalBsmtSF	int64	**SaleCondition**	object
Heating	object	**SalePrice**	int64
HeatingQC	object		

Table 1. *Variables in the dataset and corresponding variable types.*

The dependent variable that we will work with is “SalePrice”, it is of integer type and is measured in dollars. As it can be seen from the histogram below (Figure 1), the distribution seems to be positively skewed, although it resembles a normal one if we are not taking into account the outliers. There are quite many outliers (even above the upper three standard deviations threshold; there are 22 of them, and 0 below the 3std-threshold), which are highly expensive houses, and since it will probably be required to predict prices of such expensive houses also and there are many of them, I will leave them in the dataset that the model will be trained on.



Picture 1. *Distribution of the dependent variable (“SalePrice”).*

The next essential step is to deal with missing values in the training dataset, since the basic linear regression model from the Sklearn library that we have covered on the seminars does not seem to be able to process them naturally. As can be seen

from the code notebook and the table below (Table 2), for 19 variables missing values are present, and 5 of them even contain missing values more than half of the whole amount of observations (that is, more than 730 missing values). Processing them requires us to handle each variable individually. Thus, “PoolQC” missing variables simply mean a lack of pool in the house, as it is explained in dataset description on its Kaggle page. Missing values should be recoded correspondingly (I recode it as “NoPool”). The situation is similar for other categorical variables such as “MiscFeature” (lack of miscellaneous features in the house), “Alley” (lack of alley access), “Fence” (lack of fences), “MasVnrType” (lack of masonry veneer layer on walls), “FireplaceQu” (lack of fireplace), and various categorical variables connected to basement (“BsmtExposure”, “BsmtFinType2”, “BsmtFinType2”, “BsmtQual”, “BsmtCond”, and “BsmtFinType1”) and to garage (“GarageFinish”, “GarageCond”, “GarageType”, and “GarageQual”). Equal numbers of missing values for the last two groups of variables (81 for garage-related and 37-38 for basement-related) proves that missing values appeared there intentionally. For the “Electrical” feature, missing value also probably means that the house does not have electricity, but I do not think that recoding it is necessary since there is only one observation with missing value for this feature, and deleting the observation will not do the dataset any harm (it would be generally interesting how lack of electricity may influence sale price of the house, but one observation will, of course, not allow us to draw any statistically significant conclusions anyway). For a numerical garage-related variable “GarageYrBlt” (year that the garage was built) missing values also most probably signify a lack of garage in the house, and since we cannot equate it to 0 (it would spoil the calculations since the garages that were indeed build were built in the 20-21st centuries, as I have checked using *.unique()* function, although the cell was deleted from the notebook, because I considered it to be unnecessary), I equated it to the median value for the column. “MasVnrArea” and “LotFrontage” I equated to zero, since I think that missing values there also signify lack of masonry veneer and

no access to the street for the house correspondingly, and so the area for both variables is 0.

Variable name	Number of missing values	Variable name	Number of missing values
PoolQC	1453	LotShape	0
MiscFeature	1406	LandContour	0
Alley	1369	Utilities	0
Fence	1179	LotConfig	0
MasVnrType	872	LandSlope	0
FireplaceQu	690	Neighborhood	0
LotFrontage	259	Condition1	0
GarageFinish	81	BldgType	0
GarageCond	81	MSZoning	0
GarageType	81	HouseStyle	0
GarageYrBlt	81	OverallQual	0
GarageQual	81	OverallCond	0
BsmtExposure	38	YearBuilt	0
BsmtFinType2	38	YearRemodAdd	0
BsmtQual	37	RoofStyle	0
BsmtCond	37	RoofMatl	0
BsmtFinType1	37	Exterior1st	0
MasVnrArea	8	Exterior2nd	0
Electrical	1	HalfBath	0
TotalBsmtSF	0	PoolArea	0
BsmtFinSF1	0	BedroomAbvGr	0
BsmtFinSF2	0	MiscVal	0
BsmtUnfSF	0	3SsnPorch	0
HeatingQC	0	MoSold	0
Heating	0	YrSold	0
ExterCond	0	SaleType	0
CentralAir	0	SaleCondition	0
1stFlrSF	0	ScreenPorch	0
2ndFlrSF	0	WoodDeckSF	0
LowQualFinSF	0	EnclosedPorch	0

GrLivArea	0	**OpenPorchSF**	0
BsmtFullBath	0	**KitchenAbvGr**	0
BsmtHalfBath	0	**PavedDrive**	0
FullBath	0	**GarageArea**	0
Foundation	0	**GarageCars**	0
MSSubClass	0	**Fireplaces**	0
ExterQual	0	**Functional**	0
Condition2	0	**TotRmsAbvGrd**	0
LotArea	0	**KitchenQual**	0
Street	0	**SalePrice**	0

Table 2. *Variables from the general training dataset and number of corresponding missing values for each variable.*

When the missing values are dealt with, I try to deal with categorical variables so that they can be used in conducting regressions. For this, I use one-hot encoder from the Sklearn library. I do one-hot encoding before splitting the data into testing and training datasets, so that features that are created out of values present only in one dataset will not be missing from the other one. Then I also scale values of numerical variables using StandardScaler() from the same library in order for the variable to be compatible.

After the preprocessing of the general (training) dataset is completed, I split it in two, into training and testing ones. For the testing dataset, I leave 30% of all of the observations, and 60% for the training dataset. The competition also contains a separate dataset for testing, but it does not contain the values that I need to predict (values for "SalePrice"), so for preliminary testing and evaluation of the model it is crucial to split the original training dataset and only then train the model on the full training dataset when the model works well with the split one.

First, I conduct a simple Linear Regression, fitting the model with training data and making it predict values of "SalePrice" from the testing one. As can be seen from the notebook, the mean squared error (MSE) is extremely big (1.941468344844157e+28, with its square root, RMSE, being 139336583309773.92).

This needs to be fixed. The values of predictions make sense, but such a great mean squared error necessitates us to choose some other model to make predictions at least slightly close to the possible reality. In order to do that, I additionally conduct two regularized regressions — more precisely, Lasso and Ridge regressions.

Index	Alphas (Lambdas)	Zero coefficients for testing data (split from the general training dataset)	MSE for testing data (split from the general training dataset)
0	1.000000e-15	10.0	1.649562e+09
1	1.274275e-14	10.0	1.649562e+09
2	1.623777e-13	10.0	1.649562e+09
3	2.069138e-12	11.0	1.649562e+09
4	2.636651e-11	10.0	1.649562e+09
5	3.359818e-10	10.0	1.649562e+09
6	4.281332e-09	10.0	1.649562e+09
7	5.455595e-08	10.0	1.649562e+09
8	6.951928e-07	11.0	1.649562e+09
9	8.858668e-06	10.0	1.649562e+09
10	1.128838e-04	10.0	1.649560e+09
11	1.438450e-03	10.0	1.649545e+09
12	1.832981e-02	11.0	1.649349e+09
13	2.335721e-01	11.0	1.646856e+09
14	2.976351e+00	13.0	1.616526e+09
15	3.792690e+01	68.0	1.410808e+09
16	4.832930e+02	193.0	6.778866e+08
17	6.158482e+03	243.0	1.016017e+09
18	7.847600e+04	259.0	5.548070e+09
19	1.000000e+06	259.0	5.548070e+09

Table 3. *Testing various Lambdas for suitability to be used in Lasso regression.*

In order to choose the correct Lambda for Lasso regression, I run a loop in which MSEs of Lasso regressions are calculated for each element of a logarithmatised sequence of possible most-suitable Lambdas. Results of testing various Lambdas can be found in Table 3. The Lambda of 483.293 seems to be the

best choice within the chosen interval, since its MSE is the smallest among the other ones and at the same time it provides a seemingly logical amount of zero coefficients (193), or unimportant variables, for the dataset with 259 features (most of them, of course, appeared from the one-hot encoding of the categorical variables). Choosing this value of Lambda, I conduct a Lasso regression, and receive meaningful results. MSE is now 677886640.2410194, RMSE is equal to 26036.256263929718, which is quite okay, given that house prices were measured in US dollars. The predictions themselves are also reasonable.

The results for Ridge regression are slightly worse as compared to the Lasso's ones. In order to conduct Ridge regression, I first try to establish the best Lambda for the regression through the method of cross-validation and grid search. Best Lambda identified in such an automated way bring good results: MSE for this best Lambda is equal to 683536803.0556178, RMSE — to 26144.53677263412. The predictions are also meaningful. Results of Ridge regression are slightly worse than that of Lasso's, but still are significantly better than nonsense results of Linear Regression. This is proven again by values of R-squared for each model: 0.8775069990447512 for Lasso, 0.8764860239171105 for Ridge, and incomprehensible -3.5082013673410995e+18 for Linear regression. Since Lasso proves to be a bit better than Ridge, I will use Lasso in crafting predictions of houses' sale prices that are the goal of the project.

To begin working on the actual prediction of unknown values for "SalePrice", I load it, transform it into Pandas DataFrame, drop the irrelevant "Id" column and deal with missing values. In the testing data, new features have missing values, and some that contained NA's in the training dataset do not have them in the testing data. I recode them, following the logic employed in the missing values management for general training dataset. Table 4 contains data about variables and numbers of missing values present in the testing dataset. Then I concatenate training and testing datasets (with training data being cleared of missing values already in previous steps)

in order not to miss any features from either datasets while conducting one-hot encoding of the categorical variables. After this, I one-hot encode the categorical variables and scale numerical ones in a way I have done it for the training-only dataset before.

Variable name	Number of missing values	Variable name	Number of missing values
PoolQC	1456	1stFlrSF	0
MiscFeature	1408	HeatingQC	0
Alley	1352	MSSubClass	0
Fence	1169	Foundation	0
MasVnrType	894	Condition2	0
FireplaceQu	730	LotArea	0
LotFrontage	227	Street	0
GarageYrBlt	78	LotShape	0
GarageCond	78	LandContour	0
GarageQual	78	LotConfig	0
GarageFinish	78	LandSlope	0
GarageType	76	Neighborhood	0
BsmtCond	45	Condition1	0
BsmtQual	44	BldgType	0
BsmtExposure	44	ExterCond	0
BsmtFinType1	42	HouseStyle	0
BsmtFinType2	42	OverallQual	0
MasVnrArea	15	OverallCond	0
MSZoning	4	YearBuilt	0
BsmtHalfBath	2	YearRemodAdd	0
Utilities	2	RoofStyle	0
BsmtFullBath	2	RoofMatl	0
Functional	2	ExterQual	0
GarageCars	1	HalfBath	0
GarageArea	1	PoolArea	0
SaleType	1	YrSold	0
KitchenQual	1	MoSold	0
BsmtFinSF2	1	MiscVal	0

BsmtFinSF1	1	**BedroomAbvGr**	0
Exterior2nd	1	**PavedDrive**	0
TotalBsmtSF	1	**ScreenPorch**	0
BsmtUnfSF	1	**3SsnPorch**	0
Exterior1st	1	**EnclosedPorch**	0
FullBath	0	**OpenPorchSF**	0
LowQualFinSF	0	**WoodDeckSF**	0
GrLivArea	0	**KitchenAbvGr**	0
2ndFlrSF	0	**Fireplaces**	0
Heating	0	**TotRmsAbvGrd**	0
CentralAir	0	**SaleCondition**	0
Electrical	0		

Table 4. *Variables from the general testing dataset and number of corresponding missing values for each variable.*

Once preprocessing of the data is finished, I begin building the model, using training and testing datasets with now encoded categorical variables and scaled numerical variables separately. As can be seen from the notebook and Table 5, Lambda of 483.293 that I have considered to be the most suitable one in building Lasso model for split training dataset does quite well for full training dataset and testing dataset as well (in identifies quite a reasonable amount of non-important features, only slightly more than there were when conducting Lasso regression over split training dataset — 204 against 193). I use this Lambda in fitting values from training dataset for Lasso regression model and in making predictions for "SalePrice" based upon all other features from the testing dataset, and achieve meaningful results. Since there are 1459 predictions for "SalePrice" for testing dataset, I find listing all of them in this report to be irrelevant. However, I create a separate .csv file containing them, as it was required in Kaggle's task, and commit the file into GitHub repository for the project.

Index	Alphas (Lambdas)	Number of zero coefficients for testing data
0	1.000000e-15	1.0

1	1.274275e-14	0.0
2	1.623777e-13	1.0
3	2.069138e-12	0.0
4	2.636651e-11	0.0
5	3.359818e-10	0.0
6	4.281332e-09	0.0
7	5.455595e-08	0.0
8	6.951928e-07	0.0
9	8.858668e-06	0.0
10	1.128838e-04	0.0
11	1.438450e-03	0.0
12	1.832981e-02	1.0
13	2.335721e-01	1.0
14	2.976351e+00	5.0
15	3.792690e+01	75.0
16	4.832930e+02	204.0
17	6.158482e+03	244.0
18	7.847600e+04	259.0
19	1.000000e+06	259.0

Table 5. *Testing various Lambdas for suitability to be used in Lasso regression for actual testing dataset (that is, checking if Lambda used for regression over training dataset can still be used).*

In addition to the task of the project, that is, prediction of values of "SalePrice" for testing data, it is also generally important (and interesting) to look at features that are in linear relationship with the dependent variables and, thus, of importance. For the chosen Lambda for Lasso regression, there are 55 important variables, only approximately 5 of them have relatively small coefficients: these are "HalfBath" (392.849268), "EnclosedPorch" (200.050166), "3SsnPorch" (425.954409), "Foundation_Cblock" (-141.855775) and "GarageType_Attchd" (-43.430170). Thus, porches weakly influence house price if they are enclosed or 3-season ones (while more if they are screened, coefficient of 2471.464059), presence of halfbath increases the price, but also weakly, while attached garages and cement-block foundations

weakly decrease the price. Among the most significant variables (coefficient more than 15 thousands) are those related to house being located in Northridge and Northridge Heights neighborhoods (Neighborhood_NoRidge, 20695.060950, and Neighborhood_NridgHt, 21490.337199), which means that these are, probably, generally expensive neighborhoods (and they indeed are, according to the internet, what proves our model to be working correctly), and rise the interest to investigate why; and, less unexpectedly, to overall quality (OverallQual, 16871.755636), and to area of second floor (2ndFlrSF, 19462.618218). List of all important features with their corresponding coefficients can be found on Table 6. This data may be used in further analysis of the factors influencing house sale prices and in making new predictions.

	Feature	Coefficients		Feature	Coefficients
0	MSSubClass	-6134.842301	**50**	LotConfig_CulDSac	1678.986866
1	LotFrontage	-686.807876	**61**	Neighborhood_Crawfor	8680.930934
2	LotArea	2112.528335	**62**	Neighborhood_Edwards	-4675.543731
3	OverallQual	16871.755636	**70**	Neighborhood_NoRidge	20695.060950
4	OverallCond	5780.025896	**71**	Neighborhood_NridgHt	21490.337199
5	YearBuilt	12061.560801	**76**	Neighborhood_Somerst	991.476260
6	YearRemodAdd	4053.977950	**77**	Neighborhood_StoneBr	13643.909207
7	MasVnrArea	5510.320806	**81**	Condition1_Norm	8841.029553
8	BsmtFinSF1	4094.398175	**100**	HouseStyle_1Story	1535.413919
11	TotalBsmtSF	6272.466301	**103**	HouseStyle_2Story	-4681.652625
12	1stFlrSF	13105.071596	**120**	Exterior1st_BrkFace	2613.626656
13	2ndFlrSF	19462.618218	**148**	MasVnrType_None	7252.073296
15	GrLivArea	6017.303156	**152**	ExterQual_TA	-4578.021087
16	BsmtFullBath	3948.609147	**157**	Foundation_CBlock	-141.855775
18	FullBath	4532.951224	**163**	BsmtQual_Gd	-12582.826912
19	HalfBath	392.849268	**165**	BsmtQual_TA	-1433.934694
20	BedroomAbvGr	-4990.635770	**170**	BsmtExposure_Gd	9080.421894
21	KitchenAbvGr	-3967.144335	**172**	BsmtExposure_No	-6978.775736
22	TotRmsAbvGrd	4660.504252	**175**	BsmtFinType1_GLQ	929.841491
23	Fireplaces	3237.864263	**201**	KitchenQual_Gd	-10967.436489
24	GarageYrBlt	2180.362825	**202**	KitchenQual_TA	-7608.105294

25	GarageCars	8809.748688	**208**	Functional_Typ	4239.419445
27	WoodDeckSF	2212.257064	**213**	FireplaceQu_TA	-2346.498151
29	EnclosedPorch	200.050166	**214**	GarageType_Attchd	-43.430170
30	3SsnPorch	425.954409	**221**	GarageFinish_RFn	-1956.061615
31	ScreenPorch	2471.464059	**227**	GarageQual_TA	-5022.043050
32	PoolArea	-605.598079	**251**	SaleType_New	6117.671235
39	MSZoning_RM	-1527.040793	**-**		

Table 6. *Important features and their corresponding coefficients (Lasso regression).*